Praktikum 8

Pustaka / Librari SDK Arduino IDE (Sesi 2)

NRP	: 1. 3223600035
	2. 3223600040
	3. 3223600058
	4. 3223600060
Nama	: 1. Mohamad Kharizal Firdaus
	2. Rahmatulloh Syaifudin Fahmi
	3. Ihsanta Zaki Sanjaya
	4. Mochammad Ega Yudhistira
Kelas	: 2 Teknik Komputer B
Matakuliah	: Praktikum Alat Pengembangan Perangkat Lunak
Tanggal	: Kamis, 17 April 2025
Dosen	: Dr. Setiawardhana, S.T., M.T.

D4 TEKNIK KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2025

Praktikum 8

Pustaka / Librari SDK Arduino IDE (Sesi 2)

A. Tujuan

1. Mahasiswa dapat melakukan kompilasi ulang dari *.h github yang sudah ada
2. Mahasiswa dapat mengembangkan atau membuat sendiri *.h sederhana

B. Dasar Teori

Software Development Kit (SDK) merupakan sekumpulan alat pengembangan perangkat lunak yang disediakan oleh platform tertentu untuk membantu para pengembang dalam membuat aplikasi atau sistem. Dalam konteks Arduino, SDK mencakup berbagai pustaka (library), toolchain, dokumentasi, serta contoh kode yang dapat digunakan untuk berinteraksi dengan perangkat keras Arduino. SDK Arduino mempermudah pemrograman mikrokontroler dengan menyediakan abstraksi fungsi-fungsi tingkat tinggi, seperti pengendalian input/output, komunikasi serial, dan banyak lagi.

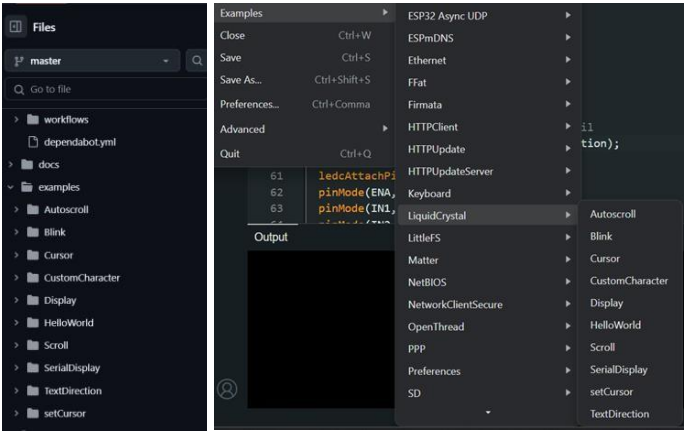
Integrated Development Environment (IDE) adalah aplikasi yang digunakan oleh pengembang untuk menulis, mengedit, mengompilasi, dan mengunggah kode ke perangkat target. Arduino IDE adalah salah satu contoh IDE yang umum digunakan untuk pemrograman mikrokontroler Arduino. Arduino IDE menyediakan antarmuka yang sederhana dan terintegrasi dengan compiler serta uploader, sehingga mempermudah proses pengembangan dan pengujian program. IDE ini juga mendukung berbagai pustaka pihak ketiga yang dapat ditambahkan untuk memperluas fungsionalitas proyek.

Reverse engineering dalam konteks pustaka atau library Arduino merujuk pada proses menganalisis kode pustaka yang sudah ada, baik dalam bentuk open-source maupun

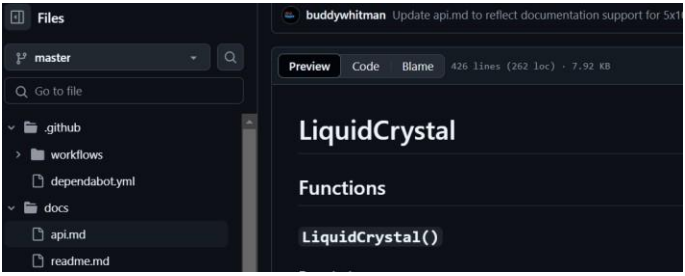
tertutup untuk memahami cara kerjanya. Tujuan dari reverse engineering ini bisa bermacam-macam, seperti untuk memodifikasi fungsi tertentu, memperbaiki bug, atau mengintegrasikan fitur yang tidak tersedia secara default. Dalam praktiknya, reverse engineering dapat melibatkan pembacaan dan penelusuran kode sumber pustaka (biasanya file .h dan .cpp), analisis struktur kelas, fungsi, serta dependensi pustaka tersebut terhadap komponen lainnya. Proses ini penting untuk memahami bagaimana library bekerja di balik layar, terutama ketika dokumentasi resmi terbatas.

C. Langkah Percobaan

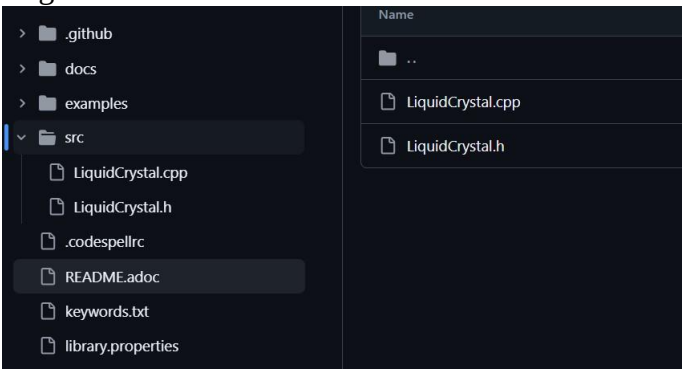
1. Perhatikan dari file-file berikut: Example dari .h



Bagian API dan workflow



Bagian source



2. Lakukan kompilasi ulang dari *.h github yang sudah ada dengan langkah-langkah:
   - Buat file header: mulailah membuat file baru dengan eksternal .h (misalnya myheader.h) yang akan berisi deklarasi fungsi
   - Gunakan include guards: include guards adalah struktur yang mencegah inklusi ganda file header. Tempatkan ini di bagian atas file, seperti contoh berikut:

```
#ifndef MYHEADER_H
#define MYHEADER_H

// Deklarasi prototipe fungsi

#endif
```

- Deklarasikan prototype fungsi: di dalam file header, deklarasikan prototype semua fungsi yang ingin anda gunakan dalam program utama.
- Sertakan dalam program utama: di dalam program utama, sertakan file header menggunakan #include “myheader.h”
- Implementasikan fungsi yang dideklarasikan dalam file header
- Gunakan fungsi: anda sekarang dapat menggunakan fungsi-fungsi ini dalam program utama dengan menyertakan file header

3. Ikuti langkah untuk melakukan kompilasi dan buat program lain secara sederhana
4. Implementasikan pula program yang ada pada modul, seperti:
   - Lingkaran

     Buat file header (lingkaran.h)

```
#ifndef LINGKARAN_H
#define LINGKARAN_H

double hitungLuasLingkaran(double radius);

#endif
```

     Implementasi fungsi (lingkaran.cpp)

```
#include "lingkaran.h"
#include <cmath>

double hitungLuasLingkaran(double radius){
    return 3.14159 * radius * radius;
}
```

     Gunakan fungsi (main.cpp)

```
#include <iostream>
#include "lingkaran.h"
```

```
int main(){
    double radius = 5.0;
    double luas = hitungLuasLingkaran(radius);
    std::cout << "Luas lingkaran dengan radius " << radius
<< " adalah " << luas << std::endl;
    return 0;
}
```

- File2

  File2.
  h

```
#ifndef file2_h
#define file2_h

#include <string>

class ClassTwo {
private:
    std::string myType;
public:
    void setType(std::string);
    std::string getType();
};

#endif
```

File2.cpp

```
#include"file2.h"

void ClassTwo::setType(std::string sType){
    myType = sType;
}

std::string ClassTwo::getType(){
    return myType;
}
```

Main-file2.cpp

```
#include <iostream>
#include "file2.h"

int main() {
    ClassTwo obj;

    obj.setType("Kelompok 7");

    std::cout << "Tipe yang diset: " << obj.getType() <<
std::endl;

    return 0;
}
```

- Reverse engineering Objectlcd.h

```
#ifndef ObjectLcd_h
#define ObjectLcd_h

#include <inttypes.h>
#include "Print.h"

#define LCD_BERSIHKAN 0x01
#define LCD_KEMBALI_AWAL 0x02
#define LCD_MODE_ENTRI 0x04
#define LCD_KONTROL_TAMPILAN 0x08
#define LCD_GESER_CURSOR 0x10
#define LCD_SET_FUNGSI 0x20
#define LCD_SET_CGRAM 0x40
#define LCD_SET_DDRAM 0x80

#define LCD_ENTRI_KANAN 0x00
#define LCD_ENTRI_KIRI 0x02
#define LCD_AUTOGESER 0x01
```

```
#define LCD_TIDAK_AUTOGESER 0x00

#define LCD_TAMPILKAN 0x04
#define LCD_SEMBUNYIKAN 0x00
#define LCD_CURSOR_NYALA 0x02
#define LCD_CURSOR_MATI 0x00
#define LCD_KEDIP 0x01
#define LCD_TIDAK_KEDIP 0x00

#define LCD_MODE8BIT 0x10
#define LCD_MODE4BIT 0x00
#define LCD_2BARIS 0x08
#define LCD_1BARIS 0x00
#define LCD_5x10 0x04
#define LCD_5x8 0x00

class LayarLCD : public Print {
public:
  LayarLCD(uint8_t rs, uint8_t enable,
           uint8_t d0, uint8_t d1, uint8_t d2, uint8_t
d3,
           uint8_t d4, uint8_t d5, uint8_t d6, uint8_t
d7);

  void mulai(uint8_t kolom, uint8_t baris, uint8_t
ukuran_karakter = LCD_5x8);
  void bersihkan();
  void keAwal();
  void tampilkan();
  void sembunyikan();
  void aturPosisi(uint8_t kolom, uint8_t baris);
  virtual size_t tulis(uint8_t);

  using Print::write;

private:
```

```
  void kirim(uint8_t, uint8_t);
  void tulis4bit(uint8_t);
  void tulis8bit(uint8_t);
  void pulsaEnable();

  uint8_t _pin_rs;
  uint8_t _pin_rw;
  uint8_t _pin_enable;
  uint8_t _pin_data[8];

  uint8_t _fungsi_tampilan;
  uint8_t _kontrol_tampilan;
  uint8_t _mode_tampilan;
  uint8_t _baris;
  uint8_t _offset_baris[4];
};

#endif
```

ObjectLcd.cpp

```
#include "ObjectLcd.h"
#include <Arduino.h>

LayarLCD::LayarLCD(uint8_t rs, uint8_t enable,
                   uint8_t d0, uint8_t d1, uint8_t d2,
uint8_t d3,
                   uint8_t d4, uint8_t d5, uint8_t d6,
uint8_t d7) {
  _pin_rs = rs;
  _pin_rw = 255;
  _pin_enable = enable;

  _pin_data[0] = d0;
  _pin_data[1] = d1;
  _pin_data[2] = d2;
```

```
  _pin_data[3] = d3;
  _pin_data[4] = d4;
  _pin_data[5] = d5;
  _pin_data[6] = d6;
  _pin_data[7] = d7;

  _fungsi_tampilan = LCD_MODE8BIT | LCD_1BARIS |
LCD_5x8;
}

void LayarLCD::mulai(uint8_t kolom, uint8_t baris,
uint8_t ukuran_karakter) {
  if (baris > 1) {
    _fungsi_tampilan |= LCD_2BARIS;
  }
  _baris = baris;
  _offset_baris[0] = 0x00;
  _offset_baris[1] = 0x40;
  _offset_baris[2] = 0x00 + kolom;
  _offset_baris[3] = 0x40 + kolom;

  if ((ukuran_karakter != LCD_5x8) && (baris == 1)) {
    _fungsi_tampilan |= LCD_5x10;
  }

  pinMode(_pin_rs, OUTPUT);
  if (_pin_rw != 255) pinMode(_pin_rw, OUTPUT);
  pinMode(_pin_enable, OUTPUT);

  for (int i = 0; i < 8; ++i) {
    pinMode(_pin_data[i], OUTPUT);
  }

  delayMicroseconds(50000);
  digitalWrite(_pin_rs, LOW);
  digitalWrite(_pin_enable, LOW);
```

```
  if (_pin_rw != 255) digitalWrite(_pin_rw, LOW);

  kirim(LCD_SET_FUNGSI | _fungsi_tampilan, LOW);
  delayMicroseconds(4500);
  kirim(LCD_SET_FUNGSI | _fungsi_tampilan, LOW);
  delayMicroseconds(150);
  kirim(LCD_SET_FUNGSI | _fungsi_tampilan, LOW);

  _kontrol_tampilan = LCD_TAMPILKAN;
  tampilkan();
  bersihkan();

  _mode_tampilan = LCD_ENTRI_KIRI |
LCD_TIDAK_AUTOGESER;
  kirim(LCD_MODE_ENTRI | _mode_tampilan, LOW);
}

void LayarLCD::bersihkan() {
  kirim(LCD_BERSIHKAN, LOW);
  delayMicroseconds(2000);
}

void LayarLCD::keAwal() {
  kirim(LCD_KEMBALI_AWAL, LOW);
  delayMicroseconds(2000);
}

void LayarLCD::aturPosisi(uint8_t kolom, uint8_t baris)
{
  if (baris >= _baris) baris = _baris - 1;
  kirim(LCD_SET_DDRAM | (kolom + _offset_baris[baris]),
LOW);
}

void LayarLCD::tampilkan() {
  _kontrol_tampilan |= LCD_TAMPILKAN;
```

```
  kirim(LCD_KONTROL_TAMPILAN | _kontrol_tampilan, LOW);
}

void LayarLCD::sembunyikan() {
  _kontrol_tampilan &= ~LCD_TAMPILKAN;
  kirim(LCD_KONTROL_TAMPILAN | _kontrol_tampilan, LOW);
}

inline size_t LayarLCD::tulis(uint8_t karakter) {
  kirim(karakter, HIGH);
  return 1;
}

void LayarLCD::kirim(uint8_t nilai, uint8_t mode) {
  digitalWrite(_pin_rs, mode);
  if (_pin_rw != 255) digitalWrite(_pin_rw, LOW);

  for (int i = 0; i < 8; i++) {
    digitalWrite(_pin_data[i], (nilai >> i) & 0x01);
  }
  pulsaEnable();
}

void LayarLCD::pulsaEnable() {
  digitalWrite(_pin_enable, LOW);
  delayMicroseconds(1);
  digitalWrite(_pin_enable, HIGH);
  delayMicroseconds(1);
  digitalWrite(_pin_enable, LOW);
  delayMicroseconds(100);
}
```

D. Hasil Percobaan

- Lingkaran

```
PS D:\Data kuliah\semester4\Alat Pengembangan Perangkat Lunak\praktikum\build>
m/build/coba.exe"
Luas Lingkaran dengan radius 5 adalah 78.5397
```

- File2

```
PS D:\Data kuliah\semester4\Alat Pengembangan Perangkat Lunak\praktikum\build
m/build/coba.exe"
Type: Kelompok 7
```

E. Analisa

Dalam program pertama, ada konsep pemisahan antara deklarasi dan definisi fungsi sudah diterapkan dengan baik. Misalnya, pada bagian perhitungan luas lingkaran, penggunaan file header lingkaran.h dan implementasinya di lingkaran.cpp mencerminkan penerapan prinsip modularitas dalam pemrograman. Pendekatan ini tidak hanya memudahkan pemeliharaan kode, tetapi juga memperjelas struktur program secara keseluruhan. Ketika dijalankan melalui main.cpp, hasil perhitungan luas lingkaran dengan radius 5 yaitu 78.5397 menunjukkan bahwa fungsi berjalan sebagaimana mestinya.

Selanjutnya, pada implementasi ClassTwo dalam file2.h dan file2.cpp, pendekatan berorientasi objek juga telah diterapkan dengan benar. Penggunaan metode setType dan getType untuk mengatur dan mengambil nilai atribut myType menunjukkan pemahaman dasar akan enkapsulasi, salah satu prinsip OOP yang penting. Ketika objek dari kelas ini dijalankan di main-file2.cpp, hasil output "Tipe yang diset: Kelompok 7" menandakan bahwa proses penyimpanan dan pengambilan data melalui metode getter dan setter bekerja dengan baik.

Bagian terakhir, yakni pada file ObjectLcd.h dan ObjectLcd.cpp, menunjukkan implementasi yang lebih kompleks terkait kendali tampilan LCD. Pendekatan reverse

engineering terhadap objek LayarLCD menampilkan penguasaan terhadap interfacing hardware dengan perangkat lunak. Fungsi-fungsi seperti tampilkan, bersihkan, dan aturPosisi disusun secara sistematis, memungkinkan pengguna untuk mengontrol layar dengan cara yang fleksibel dan efisien. Meski lebih teknis dan detail, struktur kelas yang digunakan sudah menggambarkan praktik pemrograman embedded yang baik, serta mencerminkan pemahaman mendalam terhadap kontrol low-level perangkat.

F. Kesimpulan

Penerapan prinsip modularisasi, konsep Object-Oriented Programming (OOP), dan pengendalian perangkat keras melalui program yang telah diimplementasikan mencerminkan keterkaitan langsung dengan teori Software Development Kit (SDK) dan reverse engineering. Program dikembangkan dalam lingkungan IDE yang menyediakan seluruh komponen penting seperti editor, compiler, dan debugger, sesuai dengan peran SDK dalam menyediakan perangkat lunak pendukung bagi proses pengembangan aplikasi. Pada bagian pengendalian LCD, pendekatan reverse engineering diterapkan untuk membangun ulang pustaka LayarLCD berdasarkan pemahaman terhadap antarmuka dan struktur fungsional, meskipun tanpa akses ke dokumentasi atau kode sumber asli. Melalui pengujian yang dilakukan, program terbukti berjalan sesuai dengan fungsinya, sekaligus memperkuat pemahaman terhadap bagaimana teori SDK dan reverse engineering diimplementasikan secara nyata dalam pengembangan aplikasi berbasis C++ dan platform embedded seperti Arduino.